

Analysis Report

Like

Share

839 people like this. [Sign Up](#) to see what your friends like.

↻ Share this Report

GC log file: **gc-2017_07_27-16_26.log.0**

Duration: **53 min 39 sec 863 ms**

🕒 Application waiting for resources ⚠️

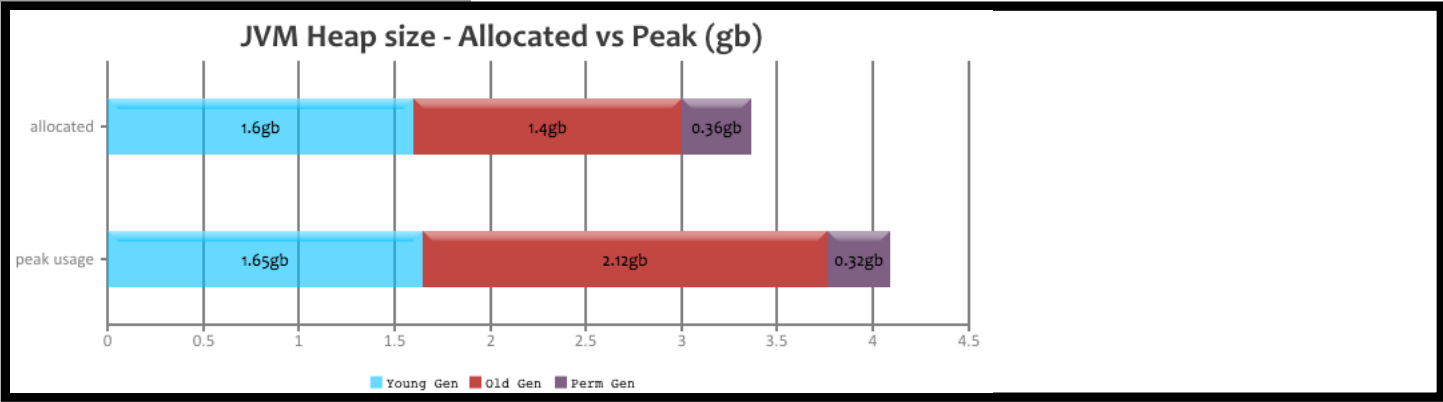
It looks like your application is waiting due to lack of compute resources (either CPU or I/O cycles). Serious production applications shouldn't be stranded because of compute resources. In **22 GC event(s)**, 'real' time took more than 'usr' + 'sys' time.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
2017-07-27T16:44:03	0.63	0.02	1.26
2017-07-27T16:44:08	0.43	0.02	1.29
2017-07-27T16:44:14	0.54	0.02	0.93
2017-07-27T16:45:06	0.38	0.01	2.24
2017-07-27T16:45:16	0.41	0.02	1.15
2017-07-27T16:45:56	0.51	0.01	0.76
2017-07-27T16:47:01	0.36	0.02	0.81
2017-07-27T16:47:08	0.56	0.05	4.43
2017-07-27T16:47:17	0.48	0.01	0.77
2017-07-27T16:47:52	0.39	0.01	1.08
2017-07-27T16:48:23	0.58	0.02	0.96
2017-07-27T16:48:32	0.57	0.01	1.34
2017-07-27T16:48:37	0.54	0.03	0.79
2017-07-27T16:53:15	0.45	0.02	1.52
2017-07-27T16:53:20	1.06	0.05	3.64
2017-07-27T16:53:27	0.45	0.02	1.16
2017-07-27T16:53:33	0.47	0.03	0.99
2017-07-27T16:53:37	0.62	0.06	4.15
2017-07-27T16:53:45	0.61	0.04	2.24
2017-07-27T16:56:17	0.58	0.01	0.88
2017-07-27T17:03:02	0.43	0.02	1.42
2 more events...			

Read our recommendations to [fix this problem \(https://blog.gceasy.io/2016/12/08/real-time-greater-than-user-and-sys-time/\)](https://blog.gceasy.io/2016/12/08/real-time-greater-than-user-and-sys-time/)

JVM Heap Size

Generation	Allocated ⓘ	Peak ⓘ
Young Generation	1.6 gb	1.65 gb
Old Generation	1.4 gb	2.12 gb
Perm Generation	372 mb	332.38 mb
Young + Old + Perm	3.36 gb	2.75 gb



Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here \(https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/\)](https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/))

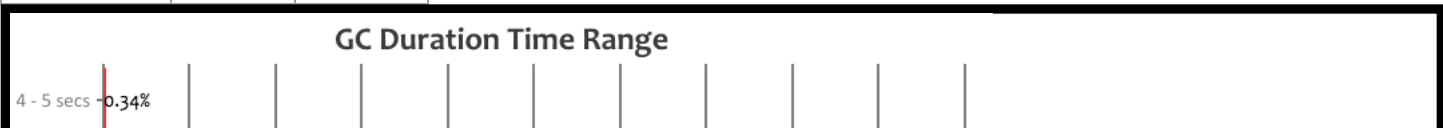
1 Throughput ⓘ : 94.774%

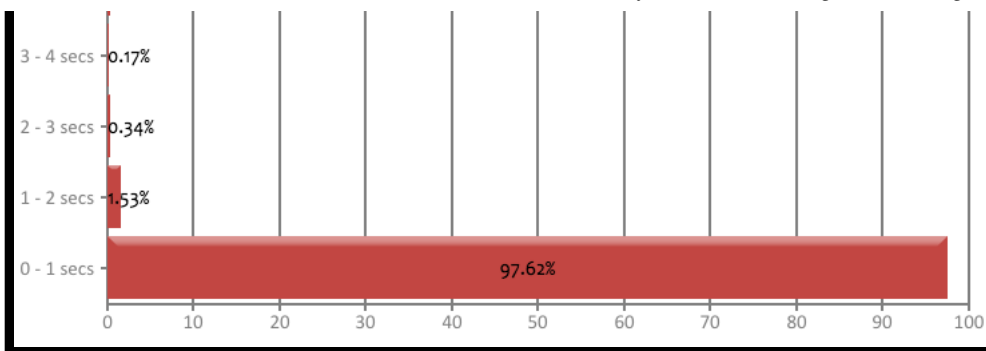
2 Latency:

Avg GC Time ⓘ	273 ms
Max GC Time ⓘ	4 sec 430 ms

GC Duration Time Range ⓘ:

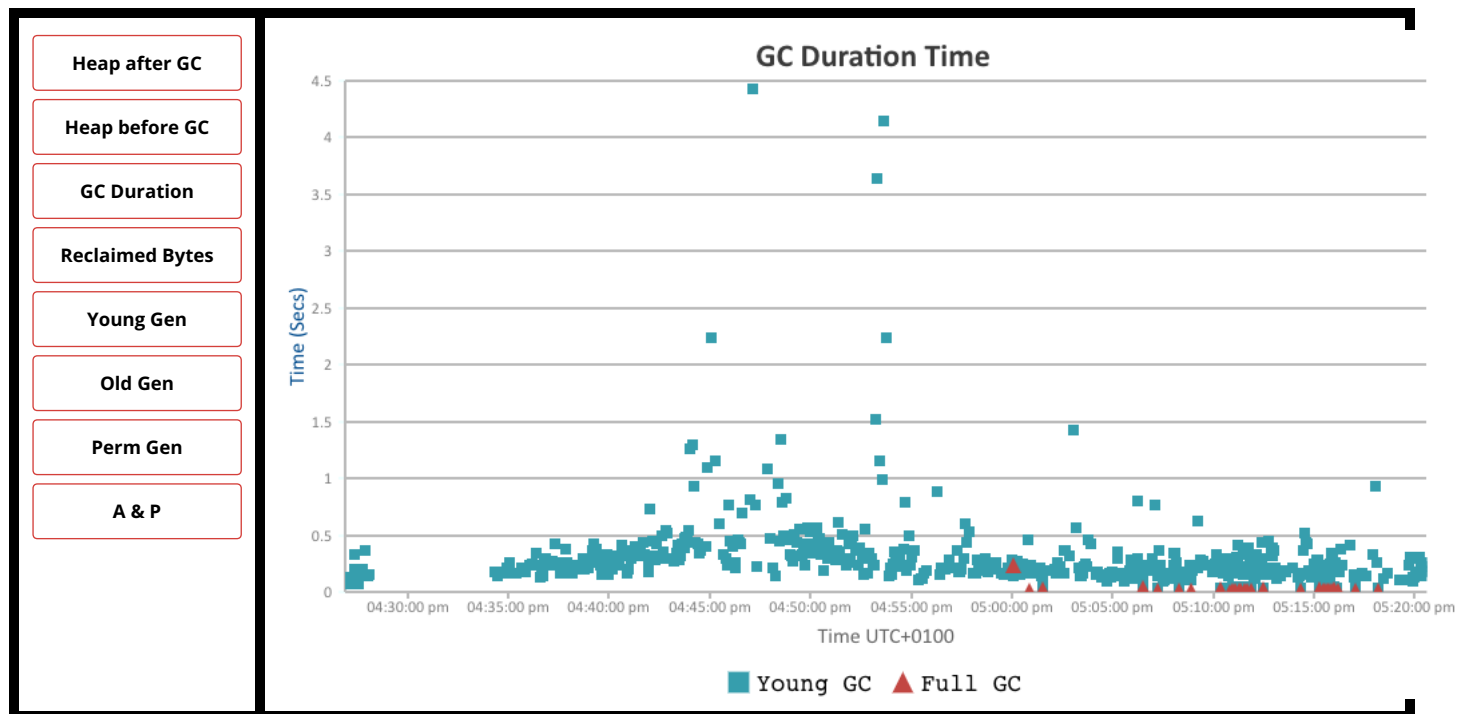
Duration (secs)	No. of GCs	Percentage
0 - 1	574	97.619%
1 - 2	9	99.15%
2 - 3	2	99.49%
3 - 4	1	99.66%
4 - 5	2	100.0%



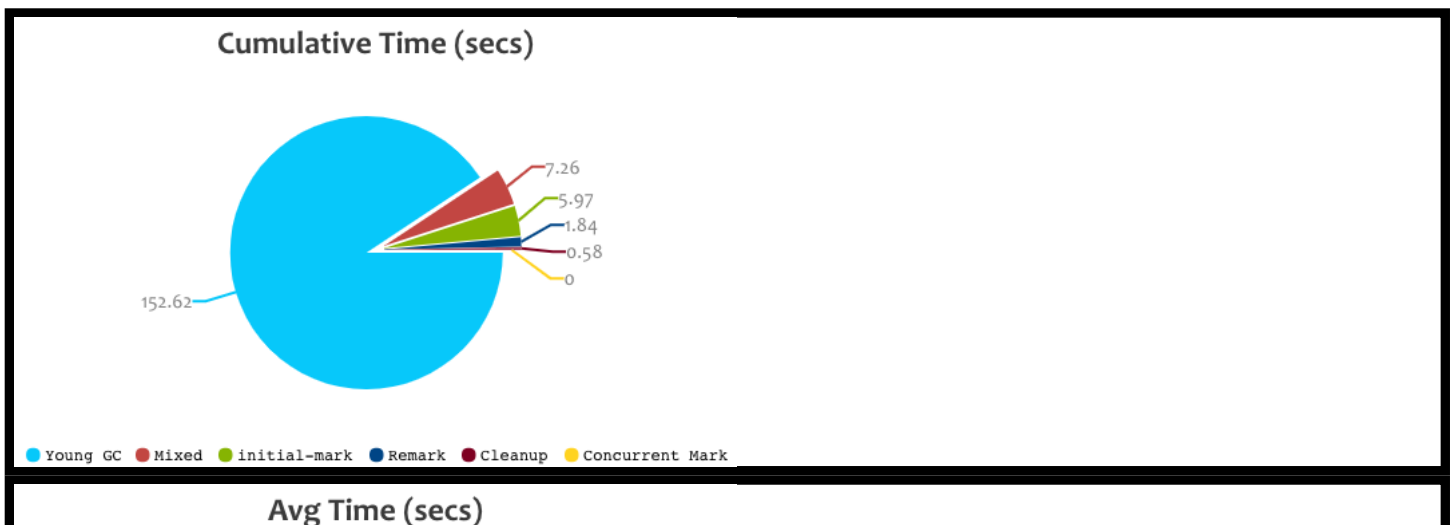


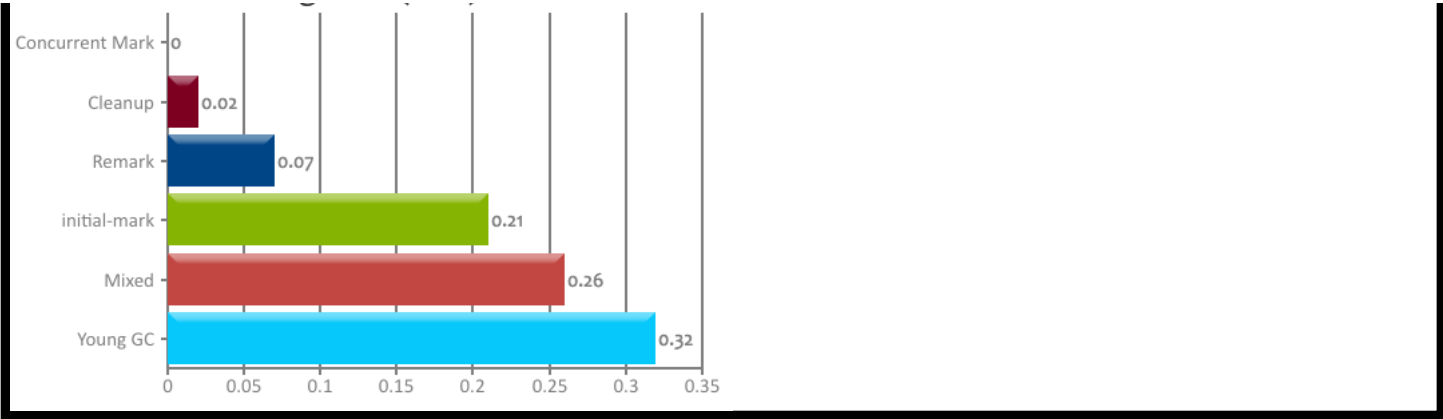
Interactive Graphs

(All graphs are zoomable)



G1 GC Statistics





	Concurrent Mark	Remark	Young GC	Cleanup	Mixed	initial-mark	Total
Count ⓘ	28	28	477	28	28	28	617
Total GC Time ⓘ	0	1 sec 840 ms	2 min 32 sec 620 ms	580 ms	7 sec 260 ms	5 sec 970 ms	2 min 48 sec 270 ms
Avg GC Time ⓘ	0	66 ms	320 ms	21 ms	259 ms	213 ms	273 ms
Avg Time std dev	0	79 ms	383 ms	43 ms	117 ms	74 ms	353 ms
Min/Max Time ⓘ	0 / 0	0 / 460 ms	0 / 4 sec 430 ms	0 / 240 ms	0 / 760 ms	0 / 410 ms	0 / 4 sec 430 ms
Avg Interval Time ⓘ	40 sec 205 ms	40 sec 189 ms	6 sec 764 ms	40 sec 186 ms	39 sec 205 ms	40 sec 266 ms	14 sec 110 ms

⚙️ Object Stats

(These are good metrics to include in your performance reports)

Total created bytes ⓘ	311.23 gb
Total promoted bytes ⓘ	4.82 gb
Avg creation rate ⓘ	98.98 mb/sec
Avg promotion rate ⓘ	1.53 mb/sec

💧 Memory Leak ⓘ

No major memory leaks.

(**Note:** there are 8 flavours of OutOfMemoryErrors (<https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf>). With GC Logs you can diagnose only 5 flavours of them(java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

⏴ Consecutive Full GC ⓘ

None.

■ Long Pause ⓘ

None.

🕒 Safe Point Duration ⓘ

(To learn more about SafePoint duration, [click here](https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/) (https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/))

Not Reported in the log.

🔍 GC Causes ⓘ

(What events caused the GCs, how much time it consumed?)

Not reported in the log.

🔄 Tenuring Summary ⓘ

Not reported in the log.

📄 Command Line Flags ⓘ

Not reported in the log.

🏆 Become a DevOps champion in your organization

(Best practises/tools)

- ✓ Use **fastThread.io** (<https://fastthread.io/>) to analyze thread dumps, core dumps and hs_err_pid files
 - ✓ Do proactive Garbage Collection analysis on all your JVMs (not just one or two) [using the GC log analysis API](https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/) 🚀 (https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/)
 - ✓ Always [rotate the GC log files](https://blog.gceasy.io/2016/11/15/rotating-gc-log-files/) (https://blog.gceasy.io/2016/11/15/rotating-gc-log-files/) to prevent critical garbage collection data loss
 - ✓ Use '[Pro](http://gceasy.io/pricing.jsp)' [version](http://gceasy.io/pricing.jsp) (http://gceasy.io/pricing.jsp) for 10x fast, unlimited, secure usage
-

Do you like this report?



GCeasy

GCeasy is the industry's first online Garbage collection log analysis tool aided by Machine Learning.